

7.3 Order of Evaluation

[This section corresponds to K&R Sec. 2.12]

When you start using the `++` and `--` operators in larger expressions, you end up with expressions which do several things at once, i.e., they modify several different variables at more or less the same time. When you write such an expression, you must be careful not to have the expression “pull the rug out from under itself” by assigning two different values to the same variable, or by assigning a new value to a variable at the same time that another part of the expression is trying to use the value of that variable.

Actually, we had already started writing expressions which did several things at once even before we met the `++` and `--` operators. The expression

```
(c = getchar()) != EOF
```

assigns `getchar`'s return value to `c`, *and* compares it to `EOF`. The `++` and `--` operators make it much easier to cram a lot into a small expression: the example

```
line[nch++] = c;
```

from the previous section assigned `c` to `line[nch]`, *and* incremented `nch`. We'll eventually meet expressions which do *three* things at once, such as

```
a[i++] = b[j++];
```

which assigns `b[j]` to `a[i]`, and increments `i`, *and* increments `j`.

If you're not careful, though, it's easy for this sort of thing to get out of hand. Can you figure out exactly what the expression

```
a[i++] = b[i++];          /* WRONG */
```

should do? I can't, and here's the important part: *neither can the compiler*. We know that the definition of postfix `++` is that the former value, before the increment, is what goes on to participate in the rest of the expression, but the expression `a[i++] = b[i++]` contains *two* `++` operators. Which of them happens first? Does this expression assign the old `i`th element of `b` to the new `i`th element of `a`, or vice versa? No one knows.

When the order of evaluation matters but is not well-defined (that is, when we can't say for sure which order the compiler will evaluate the various dependent parts in) we say that the meaning of the expression is *undefined*, and if we're smart we won't write the expression in the first place. (Why would anyone ever write an “undefined” expression? Because sometimes, the compiler happens to evaluate it in the order a programmer wanted, and the programmer assumes that since it works, it must be okay.)

For example, suppose we carelessly wrote this loop:

```
int i, a[10];
i = 0;
while(i < 10)
    a[i] = i++;          /* WRONG */
```

It looks like we're trying to set `a[0]` to 0, `a[1]` to 1, etc. But what if the increment `i++` happens

before the compiler decides which cell of the array `a` to store the (unincremented) result in? We might end up setting `a[1]` to 0, `a[2]` to 1, etc., instead. Since, in this case, we can't be sure which order things would happen in, we simply shouldn't write code like this. In this case, what we're doing matches the pattern of a `for` loop, anyway, which would be a better choice:

```
for(i = 0; i < 10; i++)
    a[i] = i;
```

Now that the increment `i++` isn't crammed into the same expression that's setting `a[i]`, the code is perfectly well-defined, and is guaranteed to do what we want.

In general, you should be wary of ever trying to second-guess the order an expression will be evaluated in, with two exceptions:

1. You can obviously assume that precedence will dictate the order in which binary operators are applied. This typically says more than just what order things happens in, but also what the expression actually *means*. (In other words, the precedence of `*` over `+` says more than that the multiplication ``happens first" in `1 + 2 * 3`; it says that the answer is 7, not 9.)
2. Although we haven't mentioned it yet, it is guaranteed that the logical operators `&&` and `||` are evaluated left-to-right, and that the right-hand side is not evaluated at all if the left-hand side determines the outcome.

To look at one more example, it might seem that the code

```
int i = 7;
printf("%d\n", i++ * i++);
```

would have to print 56, because no matter which order the increments happen in, `7*8` is `8*7` is 56. But `++` just says that the increment happens later, not that it happens immediately, so this code could print 49 (if the compiler chose to perform the multiplication first, and both increments later). And, it turns out that ambiguous expressions like this are such a bad idea that the ANSI C Standard does not require compilers to do anything reasonable with them at all. Theoretically, the above code could end up printing 42, or 8923409342, or 0, or crashing your computer.

Programmers sometimes mistakenly imagine that they can write an expression which tries to do too much at once and then predict exactly how it will behave based on ``order of evaluation." For example, we know that multiplication has higher *precedence* than addition, which means that in the expression

```
i + j * k
```

`j` will be multiplied by `k`, and then `i` will be added to the result. Informally, we often say that the multiplication happens ``before" the addition. That's true in this case, but it doesn't say as much as we might think about a more complicated expression, such as

```
i++ + j++ * k++
```

In this case, besides the addition and multiplication, `i`, `j`, and `k` are all being incremented. We can *not* say which of them will be incremented first; it's the compiler's choice. (In particular, it is *not* necessarily the case that `j++` or `k++` will happen first; the compiler might choose to save `i`'s value somewhere and increment `i` first, even though it will have to keep the old value around until after it has done the multiplication.)

In the preceding example, it probably doesn't matter which variable is incremented first. It's not too

hard, though, to write an expression where it does matter. In fact, we've seen one already: the ambiguous assignment `a[i++] = b[i++]`. We still don't know which `i++` happens first. (We can *not* assume, based on the right-to-left behavior of the `=` operator, that the right-hand `i++` will happen first.) But if we had to know what `a[i++] = b[i++]` really did, we'd have to know which `i++` happened first.

Finally, note that parentheses don't dictate overall evaluation order any more than precedence does. Parentheses override precedence and say which operands go with which operators, and they therefore affect the overall meaning of an expression, but they don't say anything about the order of subexpressions or side effects. We could not "fix" the evaluation order of any of the expressions we've been discussing by adding parentheses. If we wrote

```
i++ + (j++ * k++)
```

we still wouldn't know which of the increments would happen first. (The parentheses would force the multiplication to happen before the addition, but precedence already would have forced that, anyway.) If we wrote

```
(i++) * (i++)
```

the parentheses wouldn't force the increments to happen before the multiplication or in any well-defined order; this parenthesized version would be just as undefined as `i++ * i++` was.

There's a line from Kernighan & Ritchie, which I am fond of quoting when discussing these issues [Sec. 2.12, p. 54]:

The moral is that writing code that depends on order of evaluation is a bad programming practice in any language. Naturally, it is necessary to know what things to avoid, but if you don't know *how* they are done on various machines, you won't be tempted to take advantage of a particular implementation.

The first edition of K&R said

...if you don't know *how* they are done on various machines, that innocence may help to protect you.

I actually prefer the first edition wording. Many textbooks encourage you to write small programs to find out how your compiler implements some of these ambiguous expressions, but it's just one step from writing a small program to find out, to writing a real program which makes use of what you've just learned. But you *don't* want to write programs that work only under one particular compiler, that take advantage of the way that one compiler (but perhaps no other) happens to implement the undefined expressions. It's fine to be curious about what goes on "under the hood," and many of you will be curious enough about what's going on with these "forbidden" expressions that you'll want to investigate them, but please keep very firmly in mind that, for real programs, the very easiest way of dealing with ambiguous, undefined expressions (which one compiler interprets one way and another interprets another way and a third crashes on) is not to write them in the first place.

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995-1997 // [mail feedback](#)